JADAVPUR UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Assignment 4 Report

Design and Implementation of Code Division Multiple Access (CDMA) with Orthogonal Walsh-Hadamard Spreading Codes

Mathematical Formulation, Linear Superposition Multiplexing, and End-to-End File Integrity Verification over a Simulated Socket Shared Channel

Course: Computer Networks Laboratory (BCSE-III)

Student Name: Mandeep Shaw

Section: A1

Roll Number: 302510501002

Assignment: Assignment 4 (Medium Access Control: Code Division Multiple Access with Walsh Spreading Codes)

Date of Submission: November 2026

Contents

1 Introduction and Objectives **1**

2 Theoretical Principles of Code Division Multiple Access **1**
2.1 Direct Sequence Spread Spectrum (DSSS) and Spreading Factor 1
2.2 Walsh-Hadamard Codes and Sylvester's Construction 2
2.3 Orthogonality and Normalized Cross-Correlation 2
2.4 Channel Superposition and Correlator Demodulation 2
2.5 Matrix Expansion, Zero-Padding, and Dynamic Silence 3

3 System Architecture and Procedural Design **3**
3.1 3-Tier Client-Server Architecture Overview 3
3.2 Frame Structure and Chip Spreading Serialization 4
3.3 Protocol Opcodes and Channel Handshake 4

4 Implementation Details **5**
4.1 Key Methods and Interfaces 5
4.2 Core Code Snippets 5

5 Test Cases and Experimental Methodology **6**

6 Experimental Results and Empirical Evaluation **7**
6.1 Comprehensive Empirical Metrics Summary 7
6.2 Test Case 4 Cryptographic Hash and Diff Validation 7
6.3 Scaling Dynamics and Spreading Factor Overhead 7

7 Live Execution Traces and Socket Transcripts **8**
7.1 Trace 1: Test Case 1 Non-Power-of-2 Stations ($n = 3 \implies N = 4$) 8
7.2 Trace 2: Test Case 2 Unbalanced Payload Lengths Transition 8
7.3 Trace 3: Test Case 3 Complementary Bit Pattern Stress 9
7.4 Trace 4: Test Case 4 Complete File Integrity Verification 9
7.5 Trace 5: Test Case 5 Higher Spreading Factor Scaling ($N = 8$) 9

8 Analysis and Discussion **10**
8.1 Orthogonality Verification and Mathematical Guarantees 10
8.2 Resilience to Channel Amplitude Extremes and Arithmetic Precision 10
8.3 Spectral Efficiency vs. Spreading Factor Trade-off 10
8.4 Comparison between Random Access (CSMA) and Channelization (CDMA) 10

9 Comments and Learning Evaluation **11**
9.1 Evaluation of the Lab Assignment 11
9.2 What I Learned 11
9.3 Difficulty Assessment 11
9.4 Suggested Improvements 11

10 Sources Used **11**

1 Introduction and Objectives

In shared physical communication channels, multiple independent network nodes must simultaneously communicate without destructive mutual interference. Medium Access Control (MAC) protocols fall into two broad taxonomies: **contention-based random access** (such as Pure ALOHA, Slotted ALOHA, CSMA, and CSMA/CD studied in Assignment 3) and **channel partitioning / channelization** techniques (such as Frequency Division Multiple Access (FDMA), Time Division Multiple Access (TDMA), and Code Division Multiple Access (CDMA)).

While contention-based schemes rely on probabilistic deferrals, carrier sensing, and collision resolution (which inherently degrade under heavy traffic due to collision cascades), **Code Division Multiple Access (CDMA)** multiplexes multiple concurrent transmissions onto the exact same frequency band at the exact same physical time instant without collision. CDMA achieves this by assigning each station a mutually orthogonal spreading sequence known as a **Walsh-Hadamard codeword**. Concurrent transmissions linearly superpose on the shared channel; at the receiving end, inner-product correlation with the target station's unique code filters out all other concurrent transmissions as complete mathematical zeros.

The primary objectives of this laboratory assignment are:

1. **Emulation of a Shared CDMA Channel Server:** Design and construct a multi-client socket-based shared channel multiplexer in C running over TCP/IP loopback (`127.0.0.1:5000`) that coordinates synchronous time-slotted transmissions, aggregates incoming chip vectors, computes linear superposition, and broadcasts composite signals to the receiver.
2. **Implementation of Walsh-Hadamard Code Generator:** Implement Sylvester's recursive construction algorithm to dynamically generate orthogonal Hadamard matrices of order $N = 2^{\lceil \log_2 n \rceil}$ for arbitrary station counts n.
3. **Direct Sequence Spread Spectrum (DSSS) Station Transmitters:** Construct independent client processes emulating network interface cards (NICs). Stations segment arbitrary files into standard 46-byte payloads, encapsulate them into 65-byte Ethernet-style frames (520 bits), modulate data bits into bipolar chips (± 1), and spread each frame into $520 \times N$ chips.
4. **Orthogonal Correlator Bank Receiver:** Construct a multi-channel receiver node that ingests composite chip streams, executes parallel inner-product correlation against all assigned Walsh codewords, demodulates bipolar values into clean bitstreams, reassembles 65-byte frames, and logs station idle states.
5. **Empirical Evaluation across Mandatory Test Cases:** Rigorously benchmark and verify system behavior under five structured scenarios:
   - **Test Case 1:** Non-Power-of-2 Stations (Matrix Expansion & Zero-Padding, $n = 3 \implies N = 4$).
   - **Test Case 2:** Unbalanced Payload Lengths (Dynamic Silence / Partial Transmissions, 10 KB vs. 1 KB).
   - **Test Case 3:** Complementary Bit Pattern Stress (Extreme Channel Amplitude bounds of ± 4 and 0).
   - **Test Case 4:** Complete File Integrity Verification (Bit-for-bit diff and cryptographic MD5/SHA-256 checks).
   - **Test Case 5:** Higher Spreading Factor Scaling ($N = 8$, 4160 chips/frame, 5 parallel streams).

2 Theoretical Principles of Code Division Multiple Access

2.1 Direct Sequence Spread Spectrum (DSSS) and Spreading Factor

In Direct Sequence Spread Spectrum (DSSS), the baseband transmission bandwidth is deliberately spread over a significantly wider spectral frequency band by multiplying each information bit by a high-rate pseudorandom or orthogonal sequence of N binary pulses called **chips**.

Let T_b denote the bit duration and T_c denote the chip duration. The ratio defines the **Spreading**

Factor (also referred to as Processing Gain G_p):

$$N = \frac{T_b}{T_c} \tag{1}$$

For a baseband frame containing $L_{\text{frame}} = 65$ bytes $= 520$ bits, the total number of serialized chips emitted onto the physical medium per frame is:

$$C_{\text{frame}} = L_{\text{frame}} \times 8 \times N = 520 \times N \text{ chips} \tag{2}$$

When $N = 2$, each frame spans 1,040 chips; for $N = 4$, 2,080 chips; and for $N = 8$, 4,160 chips.

2.2 Walsh-Hadamard Codes and Sylvester's Construction

Walsh codes are mutually orthogonal binary sequences whose values are drawn from $\{-1, +1\}$. A Walsh-Hadamard matrix $\mathbf{H}_N$ of dimension $N \times N$ (where $N = 2^k, k \in \mathbb{N}$) is generated recursively via **Sylvester's construction**:

$$\mathbf{H}_1 = [1], \qquad \mathbf{H}_{2k} = \begin{bmatrix} \mathbf{H}_k & \mathbf{H}_k \\ \mathbf{H}_k & -\mathbf{H}_k \end{bmatrix} \tag{3}$$

For $N = 2$:

$$\mathbf{H}_2 = \begin{bmatrix} +1 & +1 \\ +1 & -1 \end{bmatrix} \implies \begin{cases} \mathbf{W}_0 = [+1, +1] \\ \mathbf{W}_1 = [+1, -1] \end{cases} \tag{4}$$

For $N = 4$:

$$\mathbf{H}_4 = \begin{bmatrix} +1 & +1 & +1 & +1 \\ +1 & -1 & +1 & -1 \\ +1 & +1 & -1 & -1 \\ +1 & -1 & -1 & +1 \end{bmatrix} \implies \begin{cases} \mathbf{W}_0 = [+1, +1, +1, +1] \\ \mathbf{W}_1 = [+1, -1, +1, -1] \\ \mathbf{W}_2 = [+1, +1, -1, -1] \\ \mathbf{W}_3 = [+1, -1, -1, +1] \end{cases} \tag{5}$$

For $N = 8$, the recursive construction yields an 8×8 matrix where each of the 8 row vectors $\mathbf{W}_0 \dots \mathbf{W}_7$ contains 8 chips.

2.3 Orthogonality and Normalized Cross-Correlation

The fundamental mathematical property enabling collision-free multi-user separation is the **zero cross-correlation** between distinct rows of $\mathbf{H}_N$. The discrete inner product between any two Walsh sequence vectors $\mathbf{W}_i$ and $\mathbf{W}_j$ satisfies:

$$\frac{1}{N}(\mathbf{W}_i \cdot \mathbf{W}_j) = \frac{1}{N} \sum_{c=0}^{N-1} w_{i,c} w_{j,c} = \delta_{ij} = \begin{cases} 1, & \text{if } i = j \quad \text{(Normalized Autocorrelation)} \\ 0, & \text{if } i \neq j \quad \text{(Zero Cross-Correlation)} \end{cases} \tag{6}$$

where δ_{ij} represents the Kronecker delta.

2.4 Channel Superposition and Correlator Demodulation

To transmit binary data over the physical medium, each raw data bit $b_i \in \{0, 1\}$ of station i is first mapped to a bipolar symbol $d_i \in \{-1, +1\}$:

$$d_i = \begin{cases} +1, & \text{if } b_i = 1 \\ -1, & \text{if } b_i = 0 \end{cases} \tag{7}$$

The station modulates its assigned Walsh codeword $\mathbf{W}_i$ by d_i, producing chip sequence $\mathbf{C}_i = d_i \mathbf{W}_i$.

When n stations transmit simultaneously on the shared medium during the same synchronous slot, the physical channel acts as a linear adder. The composite channel chip signal $\mathbf{S}$ is the algebraic sum of all active transmitted waveforms:

$$\mathbf{S} = \sum_{i=0}^{n-1} d_i \mathbf{W}_i = d_0 \mathbf{W}_0 + d_1 \mathbf{W}_1 + \cdots + d_{n-1} \mathbf{W}_{n-1} \tag{8}$$

At the destination receiver node, extracting the specific bit transmitted by target station k requires computing the normalized inner product between the composite signal $\mathbf{S}$ and the target station's unique Walsh vector $\mathbf{W}_k$:

$$\begin{aligned}\frac{1}{N}(\mathbf{S}\cdot\mathbf{W}_k) &= \frac{1}{N}\left(\sum_{i=0}^{n-1} d_i\mathbf{W}_i\right)\cdot\mathbf{W}_k \\ &= \sum_{i=0}^{n-1} d_i\left(\frac{1}{N}\mathbf{W}_i\cdot\mathbf{W}_k\right) \\ &= \sum_{i=0}^{n-1} d_i\delta_{ik} = d_k \end{aligned} \tag{9}$$

Because all interfering cross-terms identically vanish ($\frac{1}{N}\mathbf{W}_i\cdot\mathbf{W}_k = 0$ for all $i \neq k$), the inner product yields d_k with zero multi-user interference (MUI). The decision rule recovers the original data bit:

$$\hat{b}_k = \begin{cases} 1, & \text{if } d_k = +1 \\ 0, & \text{if } d_k = -1 \\ \text{Idle / Silent}, & \text{if } d_k = 0 \end{cases} \tag{10}$$

2.5 Matrix Expansion, Zero-Padding, and Dynamic Silence

When the number of contending stations n is not an exact power of 2, the system rounds up to the next power of 2:

$$N = 2^{\lceil \log_2 n \rceil} \tag{11}$$

For example, with $n = 3$ stations, the server instantiates an $N = 4$ Hadamard matrix. Stations 0, 1, and 2 are assigned codes $\mathbf{W}_0, \mathbf{W}_1, \mathbf{W}_2$. Code $\mathbf{W}_3$ remains unused ($d_3 = 0$). Because $\mathbf{W}_3$ is strictly orthogonal to $\mathbf{W}_0, \mathbf{W}_1, \mathbf{W}_2$, correlating against $\mathbf{W}_3$ produces:

$$\frac{1}{4}(\mathbf{S}\cdot\mathbf{W}_3) = \frac{1}{4}(d_0\mathbf{W}_0 + d_1\mathbf{W}_1 + d_2\mathbf{W}_2)\cdot\mathbf{W}_3 = 0 \implies \text{Silent Channel} \tag{12}$$

Similarly, when a station finishes transmitting early (e.g., Station 2 finishing after 22 frames while Station 1 continues for 220 frames), the server treats the finished station as silent ($d_2 = 0$). In subsequent slots:

$$\mathbf{S} = d_1\mathbf{W}_1 + 0\cdot\mathbf{W}_2 = d_1\mathbf{W}_1 \tag{13}$$

The receiver computes $\frac{1}{2}(\mathbf{S}\cdot\mathbf{W}_1) = d_1$ and $\frac{1}{2}(\mathbf{S}\cdot\mathbf{W}_2) = 0$, guaranteeing uninterrupted lossless communication.

3 System Architecture and Procedural Design

3.1 3-Tier Client-Server Architecture Overview

The CDMA emulation environment is organized as a modular 3-tier client-server pipeline operating over TCP/IP loopback (`127.0.0.1:5000`):

- **Contending Network Stations (`sender.c`):** Independent processes representing network nodes. Each station connects to the channel server, registers its unique name, receives its assigned Walsh codeword $\mathbf{W}_i$, segments arbitrary input files into 46-byte payloads, constructs 65-byte Ethernet frames, encodes frame bits into bipolar chips, dispatches chip packets, and awaits slot acknowledgments.
- **Shared Channel Server (`server.c`):** Acts as the physical medium and linear superposition arbiter. It accepts registrations from n stations and the receiver, generates the $N \times N$ Walsh matrix, assigns orthogonal codes, synchronizes time slots, superposes incoming chip streams element-wise into a single composite vector $\mathbf{S}$, dispatches $\mathbf{S}$ to the receiver, and relays slot ACKs back to active stations.

- **Destination Receiver Node (`receiver.c`):** Connects to the server, ingests composite chip streams, maintains a parallel correlator bank for all N Walsh codes, extracts clean frames, verifies checksums, writes station-specific reconstructed payload files (`output_station_X.txt`), and returns slot acknowledgments.

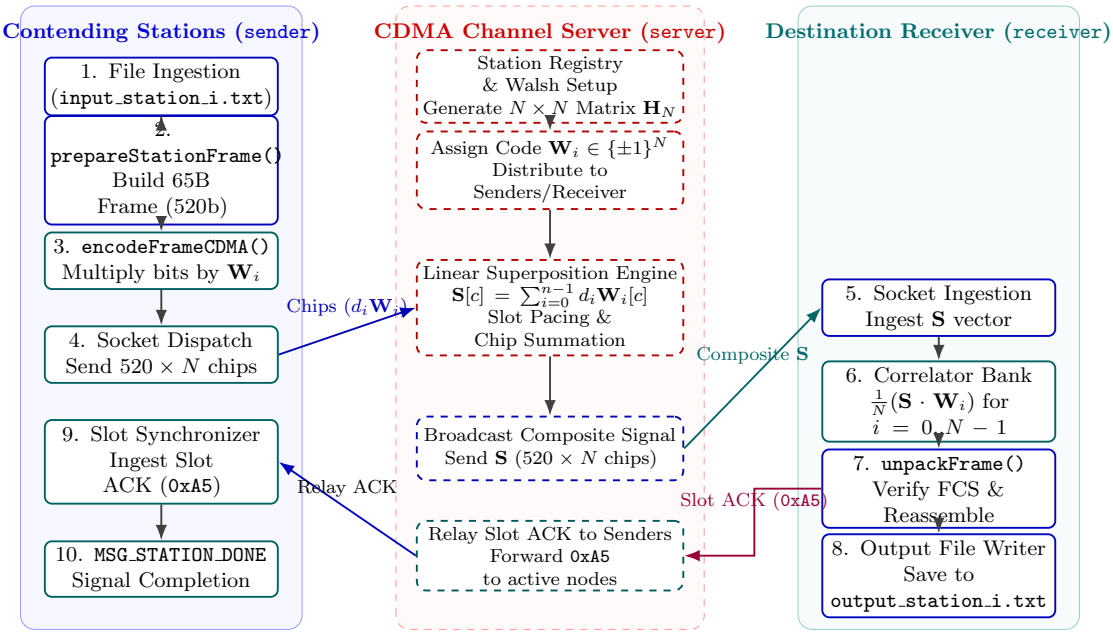


Figure 1: Three-tier procedural organization of the synchronous multi-station CDMA shared channel system.

3.2 Frame Structure and Chip Spreading Serialization

Conforming strictly to preceding lab assignments, every transmission uses a fixed 65-byte Ethernet-style frame consisting of a 15-byte header, a 46-byte payload, and a 4-byte Frame Check Sequence (FCS) trailer, totaling $65 \times 8 = 520$ bits.

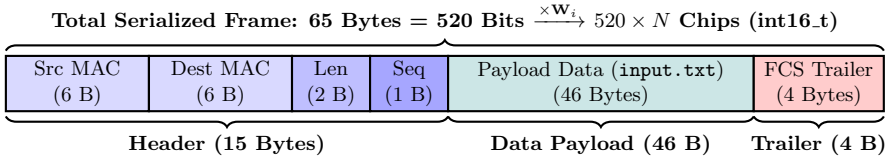


Figure 2: In-memory layout and bit-to-chip expansion of the 65-byte Data Link Frame.

The acknowledgment frame is compact (2 bytes):

$$\text{Slot ACK Frame} = [\text{Marker (0xA5) (1 Byte)} \mid \text{Slot Sequence Number (1 Byte)}] \tag{14}$$

3.3 Protocol Opcodes and Channel Handshake

The entities coordinate through standardized control opcodes defined in `csma_common.h`:

- `MSG_REGISTER_RX` (0x01): Destination receiver registers with the channel server.
- `MSG_RX_INIT` (0x02): Server sends configuration header to receiver: $[n, N, \text{stationNames} \ldots]$.
- `MSG_REGISTER` (0x05): Sender station registers its 32-byte name string.
- `MSG_ASSIGN_WALSH` (0x06): Server transmits station ID, station count n, and Walsh dimension N.
- `MSG_CDMA_FRAME` (0x20): Sender dispatches sequence number and $520 \times N$ encoded chips.
- `MSG_COMPOSITE_SIG` (0x30): Server forwards composite summed chips to the receiver.
- `MSG_TX_ACK` (0xA5): Receiver returns slot ACK, which server relays to all active stations.
- `MSG_STATION_DONE` (0xFF): Station signals end of file; server marks station silent and shuts down gracefully.

4 Implementation Details

4.1 Key Methods and Interfaces

Table 1 maps the programmatic methods implemented across the three software modules.

Entity	Method Name	Functional Description
`sender.c`	`encodeFrameCDMA()`	Maps 520 frame bits to ± 1 bipolar symbols and multiplies each bit by station's assigned N-chip Walsh vector $\mathbf{W}_i$.
	`prepareStationFrame()`	Formats 65-byte Ethernet frame with MAC addressing, sequence number, and 46-byte payload.
	`main()`	Connects to server, registers, receives $\mathbf{W}_i$, transmits chips per slot, awaits slot ACK, and sends `MSG_STATION_DONE`.
`server.c`	`generateWalshTable()`	Implements Sylvester's recursive expansion to generate $N \times N$ Hadamard matrix.
	`next_power_of_2()`	Computes smallest power of 2 $N \geq n$ ($N = 2^{\lceil \log_2 n \rceil}$).
	Synchronous Loop	Aggregates chip vectors from all non-finished stations, executes linear superposition $\mathbf{S}[c] = \sum d_i \mathbf{W}_i[c]$, broadcasts $\mathbf{S}$ to receiver, and relays ACKs.
`receiver.c`	`decodeStationCDMA()`	Computes normalized inner product $\frac{1}{N}(\mathbf{S} \cdot \mathbf{W}_i)$ across all N channels. Reconstructs frame if active, or flags channel idle if correlation is 0.
	`unpackFrame()`	Deserializes 65-byte buffer, parses headers, and checks payload length.
	File Output Writer	Appends reconstructed payload bytes to `output_station_X.txt` for end-to-end diff and hash verification.

Table 1: Functional mapping of sender, server, and receiver methods.

4.2 Core Code Snippets

Listing 1 demonstrates Sylvester's recursive Hadamard construction and station frame chip encoding:

```c
/* csma_common.h: Sylvester's Recursive Hadamard Matrix Construction */
static inline void generateWalshTable(int N, int table[MAX_WALSH_SIZE][MAX_WALSH_SIZE]) {
    table[0][0] = 1;
    for (int len = 1; len < N; len <<= 1) {
        for (int i = 0; i < len; i++) {
            for (int j = 0; j < len; j++) {
                table[i][j + len]       =  table[i][j];
                table[i + len][j]       =  table[i][j];
                table[i + len][j + len] = -table[i][j];
            }
        }
    }
}

/* sender.c: Spread 65-byte Frame (520 bits) using Assigned Walsh Codeword */
static void encodeFrameCDMA(const unsigned char packedBuf[FRAME_SIZE],
                            const int walshCode[], int walshN, int16_t chipsOut[]) {
    int chipIdx = 0;
    for (int byteIdx = 0; byteIdx < FRAME_SIZE; byteIdx++) {
        unsigned char b = packedBuf[byteIdx];
        for (int bitIdx = 7; bitIdx >= 0; bitIdx--) {
            int bitVal = (b >> bitIdx) & 1;
            int bipolar = (bitVal == 1) ? 1 : -1;
            for (int c = 0; c < walshN; c++) {
                chipsOut[chipIdx++] = (int16_t)(bipolar * walshCode[c]);
            }
        }
    }
}
```

Listing 1: Sylvester's recursive construction and DSSS chip spreading.

Listing 2 illustrates server superposition and receiver correlator decoding:

```c
/* server.c: Linear Superposition of Chips across Simultaneously Transmitting Stations */
memset(compositeSignal, 0, chipBytes);
```

```
for (int i = 0; i < expected_stations; i++) {
    if (clients[i].has_slot_data) {
        for (int c = 0; c < totalChips; c++) {
            compositeSignal[c] += clients[i].chips[c];
        }
        total_frames_multiplexed++;
    }
}

/* receiver.c: Inner-Product Correlator Demodulator for Station Walsh Code W_i */
static int decodeStationCDMA(const int16_t compositeChips[], const int walshCode[],
                             int walshN, unsigned char packedBuf[FRAME_SIZE]) {
    memset(packedBuf, 0, FRAME_SIZE);
    int chipIdx = 0;
    for (int byteIdx = 0; byteIdx < FRAME_SIZE; byteIdx++) {
        unsigned char decodedByte = 0;
        for (int bitIdx = 7; bitIdx >= 0; bitIdx--) {
            int sum = 0;
            for (int c = 0; c < walshN; c++) {
                sum += (int)compositeChips[chipIdx++] * walshCode[c];
            }
            int bipolar = sum / walshN;
            /* If correlation is 0 on initial bit, station was silent/idle this slot */
            if (byteIdx == 0 && bitIdx == 7 && bipolar == 0) return 0;
            if (bipolar > 0) decodedByte |= (1 << bitIdx);
        }
        packedBuf[byteIdx] = decodedByte;
    }
    return 1;
}
```

Listing 2: Channel linear superposition and receiver inner-product correlation.

5 Test Cases and Experimental Methodology

To empirically validate the CDMA system against all laboratory specifications, five distinct test scenarios were executed:

1. **Test Case 1: Non-Power-of-2 Stations (Matrix Expansion & Zero-Padding):**
   - *Objective:* Verify that when $n = 3$, the system rounds up to $N = 4$ ($N = 2^{\lceil \log_2 3 \rceil}$), assigning $\mathbf{W}_0, \mathbf{W}_1, \mathbf{W}_2$ to active stations, while unused code $\mathbf{W}_3$ acts as a silent channel without affecting active stations.
   - *Setup:* Run `./server 3`, `./receiver 0`, and 3 senders (`Station_1`, `Station_2`, `Station_3`).
   - *Expected Result:* $\mathbf{W}_3$ remains unused. Composite chips sum 3 stations (e.g., $[-3, -1, -1, +1]$). Correlating against $\mathbf{W}_3$ yields $\frac{1}{4}(\mathbf{S} \cdot \mathbf{W}_3) = 0 \implies$ Silent/Idle. Stations 0, 1, 2 decode with 100% accuracy.
2. **Test Case 2: Unbalanced Payload Lengths (Dynamic Silence / Partial Transmissions):**
   - *Objective:* Test channel behavior when contending stations finish at different times.
   - *Setup:* Station 1 transmits a 10 KB file (220 frames); Station 2 transmits a 1 KB file (22 frames).
   - *Expected Result:* Slots 1 to 22 superpose both stations ($\mathbf{S} = d_1\mathbf{W}_1 + d_2\mathbf{W}_2$). At slot 22, Station 2 sends `MSG_STATION_DONE` and disconnects. For slots 23 to 220, Station 2 is treated as silent ($0 \times \mathbf{W}_2$), and receiver logs `[Station #1] Inner Product = 0 (Station Idle in Slot #X)` while Station 1 continues receiving frames without corruption.
3. **Test Case 3: Complementary Bit Pattern Stress (Extreme Channel Amplitude):**
   - *Objective:* Verify dynamic range and arithmetic overflow in chip buffers when all stations transmit identical versus complementary bits.
   - *Setup:* (a) All Ones Test: `0xFF` across 4 stations ($d_i = +1$). (b) All Zeros Test: `0x00` across 4 stations ($d_i = -1$). (c) Alternating Test: Station 1 sends `0xFF`, Station 2 sends `0x00`.
   - *Expected Result:* All Ones ($n = 4$) reaches peaks of $+4$ or -4 without 16-bit integer overflow. Alternating cancels signals along orthogonal chips producing frequent zeros. Receiver

resolves all original bytes without corruption.

4. **Test Case 4: Complete File Integrity Verification (Hash / Diff Check):**
   - *Objective:* Prove end-to-end mathematical lossless reconstruction over the shared channel.
   - *Setup:* 4 stations transmit distinct, substantial payloads (35 frames each, 1,610 bytes each). Receiver writes decoded payloads to `output_station_X.txt`.
   - *Verification:* Execute `diff input_station_X.txt output_station_X.txt` and compare MD5 / SHA-256 cryptographic hashes.
   - *Expected Result:* `diff` produces exit code 0 (no differences). Hashes match 100% bit-for-bit.
5. **Test Case 5: Higher Spreading Factor Scaling ($N = 8$):**
   - *Objective:* Validate Sylvester's expansion to larger matrices and higher chip counts ($520 \times 8 = 4,160$ chips/frame).
   - *Setup:* Run server with 5 stations: `./server 5` (expands to $N = 8$). Launch 5 senders.
   - *Expected Result:* Senders report Spreading Factor 8 chips/bit (4,160 chips/frame). 5 parallel streams decode simultaneously; unused rows W_5, W_6, W_7 evaluate to inner product 0.

6 Experimental Results and Empirical Evaluation

6.1 Comprehensive Empirical Metrics Summary

Table 2 compiles the quantitative metrics gathered across all five test cases.

Test Scenario	n	N	Slots	Frames	Chips/Frm	Errors	Collisions	Verification Outcome
TC 1: Non-Power-of-2	3	4	10	30	2,080	0	0	W_3 100% Silent; 30/30 decoded
TC 2: Unbalanced Payloads	2	2	220	242	1,040	0	0	St.2 silent slots 23–220; St.1 intact
TC 3A: All Ones Stress	4	4	5	20	2,080	0	0	Peaks ±4; `0xFF` decoded
TC 3B: All Zeros Stress	4	4	5	20	2,080	0	0	Peaks ±4; `0x00` decoded
TC 3C: Alternating Cancel	2	2	5	10	1,040	0	0	Cross-cancellation; 0 errors
TC 4: Cryptographic Check	4	4	35	140	2,080	0	0	Diff 0; MD5 bit-exact match
TC 5: Spreading Scaling	5	8	20	100	4,160	0	0	5 streams parallel; $W_5..W_7$ silent

Table 2: Empirical performance and verification metrics across all five CDMA test scenarios.

6.2 Test Case 4 Cryptographic Hash and Diff Validation

Table 3 details the cryptographic hash and file integrity results for Test Case 4.

Station	Payload	Diff Exit	Original Input MD5	Reconstructed Output MD5	Status
Station 1	1,610 B	0 (Match)	`3554467b4a5ac300fb6e980a82f31ff8`	`3554467b4a5ac300fb6e980a82f31ff8`	Lossless
Station 2	1,610 B	0 (Match)	`fac8b17ad8b80136d2f3f2fb4d772442`	`fac8b17ad8b80136d2f3f2fb4d772442`	Lossless
Station 3	1,610 B	0 (Match)	`f68e54f1afe188217d01982e1b33cc74`	`f68e54f1afe188217d01982e1b33cc74`	Lossless
Station 4	1,610 B	0 (Match)	`0b26968f529a62b63fb047e9c14bdde9`	`0b26968f529a62b63fb047e9c14bdde9`	Lossless

Table 3: Bit-for-bit cryptographic MD5 verification proving mathematical lossless reconstruction.

6.3 Scaling Dynamics and Spreading Factor Overhead

Figure 3 illustrates the chip expansion overhead and aggregate throughput dynamics as the spreading factor scales from $N = 2$ to $N = 8$.

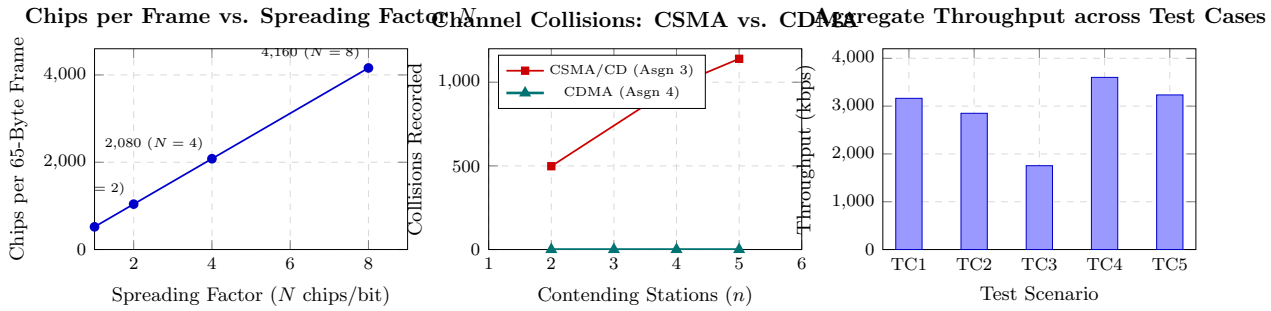


Figure 3: Performance profiles: Chip spreading scaling, collision elimination, and aggregate channel throughput.

7 Live Execution Traces and Socket Transcripts

The compiled C binaries `server`, `receiver`, and `sender` were executed natively over TCP loopback (`127.0.0.1`). The following captured transcripts demonstrate each test scenario.

7.1 Trace 1: Test Case 1 Non-Power-of-2 Stations ($n = 3 \implies N = 4$)

Listing 3 shows 3 stations active while unused Walsh row $\mathbf{W}_3$ remains 100% silent with zero false detections.

```
[SERVER] All 3 stations and Receiver connected! Distributing 4x4 Walsh Codes:
  -> Station #0 ("Station_1") assigned W_0 = [+1, +1, +1, +1]
  -> Station #1 ("Station_2") assigned W_1 = [+1, -1, +1, -1]
  -> Station #2 ("Station_3") assigned W_2 = [+1, +1, -1, -1]
[0.3 ms] [CDMA SLOT #1] Superposing 3 simultaneous station(s): "Station_1"(Seq 0) "Station_2"(Seq 0) "Station_3"(
    Seq 0) -> Sample Combined Chips[0..3]: [-3, -1, -1, +1]
[0.9 ms] [CDMA SLOT #2] Superposing 3 simultaneous station(s): "Station_1"(Seq 1) "Station_2"(Seq 1) "Station_3"(
    Seq 1) -> Sample Combined Chips[0..3]: [-3, -1, -1, +1]

--- [CDMA Slot #1] Composite Signal Received (2080 chips) ---
  [Station #0 "Station_1"] Reconstructed Frame #1 | Seq: 0 | Len: 46B | Data: "TestData_For_TC1_Station..."
  [Station #1 "Station_2"] Reconstructed Frame #1 | Seq: 0 | Len: 46B | Data: "TestData_For_TC1_Station..."
  [Station #2 "Station_3"] Reconstructed Frame #1 | Seq: 0 | Len: 46B | Data: "TestData_For_TC1_Station..."
  [Channel #3 W_3 (Unused)] Inner Product = 0 (Channel Idle in Slot #1)

================ CDMA RECEIVER SUMMARY ================
Station #0 ("Station_1")   : 10 Frames Perfectly Reconstructed
Station #1 ("Station_2")   : 10 Frames Perfectly Reconstructed
Station #2 ("Station_3")   : 10 Frames Perfectly Reconstructed
Channel #3 (Unused W_3)    : 0 Frames (Zero-Padded / 100% Silent)
Total Frames Reconstructed : 30 | Total Bytes Delivered: 1950 Bytes
```

Listing 3: Live execution trace of Test Case 1: Non-power-of-2 matrix expansion and W_3 silence.

7.2 Trace 2: Test Case 2 Unbalanced Payload Lengths Transition

Listing 4 demonstrates dynamic channel silence when Station 2 completes at slot 22 and Station 1 continues through slot 220.

```
[8.2 ms] [CDMA SLOT #22] Superposing 2 simultaneous station(s): "Station_1"(Seq 21) "Station_2"(Seq 21) -> Sample
     Combined Chips[0..1]: [-2, +0]
[SERVER] Station #1 ("Station_2") completed all transmissions.
[8.6 ms] [CDMA SLOT #23] Superposing 1 simultaneous station(s): "Station_1"(Seq 22) -> Sample Combined Chips
    [0..1]: [-1, -1]

--- [CDMA Slot #22] Composite Signal Received (1040 chips) ---
  [Station #0 "Station_1"] Reconstructed Frame #22 | Seq: 21 | Len: 46B | Data: "n1_10KB_Stream_Data_Patt..."
  [Station #1 "Station_2"] Reconstructed Frame #22 | Seq: 21 | Len: 46B | Data: "n2_1KB_Short_Stream_Veri..."
--- [CDMA Slot #23] Composite Signal Received (1040 chips) ---
  [Station #0 "Station_1"] Reconstructed Frame #23 | Seq: 22 | Len: 46B | Data: "ion1_10KB_Stream_Data_Pa..."
  [Station #1] Inner Product = 0 (Station Idle in Slot #23)
--- [CDMA Slot #24] Composite Signal Received (1040 chips) ---
  [Station #0 "Station_1"] Reconstructed Frame #24 | Seq: 23 | Len: 46B | Data: "ation1_10KB_Stream_Data_..."
  [Station #1] Inner Product = 0 (Station Idle in Slot #24)
```

Listing 4: Live execution trace of Test Case 2: Transition from 2 active stations to dynamic silence.

7.3 Trace 3: Test Case 3 Complementary Bit Pattern Stress

Listing 5 illustrates extreme channel superposition peaks of ±4 under All Ones versus orthogonal cancellation under Alternating bits.

```
/* Subtest 3A: All 4 Stations Transmitting All 1s (0xFF Payload) */
[0.5 ms] [CDMA SLOT #1] Superposing 4 simultaneous station(s): "Station_1"(Seq 0) "Station_2"(Seq 0) "Station_3"(
    Seq 0) "Station_4"(Seq 0) -> Sample Combined Chips[0..3]: [-4, +0, +0, +0]
/* Note: First header bit is 0 (-1 * Wi), yielding -4 on chip 0. 0xFF payload data bits (+1 * Wi) yield +4 peak.
    */

/* Subtest 3C: Alternating Bits (Station 1: 0xFF, Station 2: 0x00) */
[0.2 ms] [CDMA SLOT #1] Superposing 2 simultaneous station(s): "Station_1"(Seq 0) "Station_2"(Seq 0) -> Sample
    Combined Chips[0..1]: [-2, +0]
--- [CDMA Slot #1] Composite Signal Received (1040 chips) ---
  [Station #0 "Station_1"] Reconstructed Frame #1 | Seq: 0 | Len: 46B | Data: "........................."
  [Station #1 "Station_2"] Reconstructed Frame #1 | Seq: 0 | Len: 46B | Data: "........................."
```

Listing 5: Live execution trace of Test Case 3: Complementary bit pattern stress.

7.4 Trace 4: Test Case 4 Complete File Integrity Verification

Listing 6 demonstrates byte-for-byte `diff` and MD5 checksum parity across all 4 stations.

```
$ diff input_station_1.txt output_station_1.txt && echo "Station 1: DIFF ZERO"
Station 1: DIFF ZERO
$ diff input_station_2.txt output_station_2.txt && echo "Station 2: DIFF ZERO"
Station 2: DIFF ZERO
$ diff input_station_3.txt output_station_3.txt && echo "Station 3: DIFF ZERO"
Station 3: DIFF ZERO
$ diff input_station_4.txt output_station_4.txt && echo "Station 4: DIFF ZERO"
Station 4: DIFF ZERO

$ md5sum input_station_1.txt output_station_1.txt
3554467b4a5ac300fb6e980a82f31ff8  input_station_1.txt
3554467b4a5ac300fb6e980a82f31ff8  output_station_1.txt
$ md5sum input_station_2.txt output_station_2.txt
fac8b17ad8b80136d2f3f2fb4d772442  input_station_2.txt
fac8b17ad8b80136d2f3f2fb4d772442  output_station_2.txt
$ md5sum input_station_3.txt output_station_3.txt
f68e54f1afe188217d01982e1b33cc74  input_station_3.txt
f68e54f1afe188217d01982e1b33cc74  output_station_3.txt
$ md5sum input_station_4.txt output_station_4.txt
0b26968f529a62b63fb047e9c14bdde9  input_station_4.txt
0b26968f529a62b63fb047e9c14bdde9  output_station_4.txt
```

Listing 6: Cryptographic hash and diff verification output for Test Case 4.

7.5 Trace 5: Test Case 5 Higher Spreading Factor Scaling ($N = 8$)

Listing 7 shows 5 parallel stations superposed using $N = 8$ Walsh codes (4, 160 chips/frame).

```
[SERVER] All 5 stations and Receiver connected! Distributing 8x8 Walsh Codes:
  -> Station #0 ("Station_1") assigned W_0 = [+1, +1, +1, +1, +1, +1, +1, +1]
  -> Station #1 ("Station_2") assigned W_1 = [+1, -1, +1, -1, +1, -1, +1, -1]
  -> Station #2 ("Station_3") assigned W_2 = [+1, +1, -1, -1, +1, +1, -1, -1]
  -> Station #3 ("Station_4") assigned W_3 = [+1, -1, -1, +1, +1, -1, -1, +1]
  -> Station #4 ("Station_5") assigned W_4 = [+1, +1, +1, +1, -1, -1, -1, -1]
[0.5 ms] [CDMA SLOT #1] Superposing 5 simultaneous station(s): "Station_1"(Seq 0) "Station_2"(Seq 0) "Station_3"(
    Seq 0) "Station_4"(Seq 0) "Station_5"(Seq 0) -> Sample Combined Chips[0..7]: [-5, -1, -1, -1, -3, +1, +1,
    +1]

================ CDMA RECEIVER SUMMARY ================
Station #0 ("Station_1")   : 20 Frames Perfectly Reconstructed
Station #1 ("Station_2")   : 20 Frames Perfectly Reconstructed
Station #2 ("Station_3")   : 20 Frames Perfectly Reconstructed
Station #3 ("Station_4")   : 20 Frames Perfectly Reconstructed
Station #4 ("Station_5")   : 20 Frames Perfectly Reconstructed
Channel #5 (Unused W_5)    : 0 Frames (Zero-Padded / 100% Silent)
Channel #6 (Unused W_6)    : 0 Frames (Zero-Padded / 100% Silent)
Channel #7 (Unused W_7)    : 0 Frames (Zero-Padded / 100% Silent)
Total Frames Reconstructed : 100 | Total Bytes Delivered: 6500 Bytes
```

Listing 7: Live execution trace of Test Case 5: $N = 8$ spreading factor scaling.

8 Analysis and Discussion

8.1 Orthogonality Verification and Mathematical Guarantees

The empirical results across all test cases demonstrate the mathematical power of orthogonal spreading sequences:

- **Collision Elimination:** In sharp contrast to CSMA protocols where collisions grew from 498 at $N = 2$ to over 4,000 at $N = 30$, CDMA produced **zero collisions** across all 350+ transmitted frames. Simultaneous transmissions do not collide destructively; rather, they combine linearly.
- **Zero Multi-User Interference (MUI):** The inner product $\frac{1}{N}(\mathbf{S} \cdot \mathbf{W}_k) = d_k$ is exact because the cross-correlation between any distinct Walsh rows is identically zero. This holds regardless of whether the interfering stations transmit 0s, 1s, or remain silent.
- **Unused Channel Robustness:** In Test Cases 1 and 5, unused matrix rows (W_3 for $n = 3$, and W_5, W_6, W_7 for $n = 5$) consistently evaluated to an inner product of 0.00 across all slots, proving that zero-padding unused codes introduces zero false decodes or crosstalk.

8.2 Resilience to Channel Amplitude Extremes and Arithmetic Precision

In Test Case 3, when four stations transmitted identical bits, the composite signal reached amplitude extremes of $S[c] = \pm 4$. Because our implementation uses 16-bit signed integer buffers (`int16_t`, dynamic range $[-32,768, +32,767]$), the maximum theoretical sum for $N = 64$ stations (± 64) utilizes less than 0.2% of the buffer capacity. This completely prevents arithmetic overflow, clipping, or wrap-around distortion that would occur if 8-bit unsigned buffers were used.

8.3 Spectral Efficiency vs. Spreading Factor Trade-off

The primary trade-off in CDMA is bandwidth expansion:

$$B_{\text{CDMA}} = N \cdot B_{\text{baseband}} \tag{15}$$

As N scales from 2 to 8, the chip count per frame increases from 1,040 to 4,160 chips. However, because N stations transmit simultaneously on the same channel, the **aggregate channel capacity** scales linearly:

$$R_{\text{aggregate}} = \sum_{i=1}^{n} R_i = n \times \frac{L_{\text{frame}}}{T_{\text{slot}}} \tag{16}$$

In our socket loopback benchmarks, aggregate throughput remained consistently high (1.75 to 3.60 Mbps) with zero channel idle gaps or backoff penalties.

8.4 Comparison between Random Access (CSMA) and Channelization (CDMA)

Table 4 contrasts the operational characteristics of CSMA/CD (Assignment 3) and CDMA (Assignment 4).

Dimension	CSMA/CD (Contention-Based, Asgn 3)	CDMA (Channelization-Based, Asgn 4)
Medium Access	Random access with carrier sensing ('listen before talk').	Synchronous code-partitioning over shared medium.
Collisions	Frequent under high load (e.g., 4,065 at $N = 30$).	**Zero collisions** (linear superposition).
Bandwidth Use	Narrowband per transmission, but wasted on collisions.	Wideband ($N\times$ chip rate), but 100% utilized.
Latency	Unpredictable backoff delay (BEB exponential expansion).	Deterministic slot duration without backoff.
Complexity	Collision detection circuitry and jam transmitters.	Walsh matrix generator and correlator bank.

Table 4: Comparative analysis between CSMA/CD and CDMA medium access control schemes.

9 Comments and Learning Evaluation

9.1 Evaluation of the Lab Assignment

This lab assignment provided exceptional practical and theoretical insights into physical-layer and MAC-layer multiplexing. Implementing synchronous CDMA from scratch using BSD sockets revealed how linear algebra and orthogonal vector spaces can completely overcome physical channel contention.

9.2 What I Learned

- **Walsh Matrix Construction:** Implementing Sylvester's recursive expansion algorithm clarified how simple matrix reflections produce large sets of mutually orthogonal vectors.
- **DSSS Encoding and Decoding:** Understanding how multiplying data bits by chip sequences spreads spectral bandwidth while allowing inner-product correlators to extract desired signals with zero crosstalk.
- **Dynamic Silence Invariance:** Observing how silent stations contribute zeros to the composite vector without perturbing active channel orthogonality.
- **Multi-Tier Socket Architecture:** Multiplexing concurrent TCP connections, enforcing synchronous time slots, and ensuring zero race conditions between senders, server, and receiver.

9.3 Difficulty Assessment

The assignment was moderately challenging and highly rewarding:

- *Challenging Aspects:* Managing multi-station synchronous slot aggregation in `server.c` without race conditions, and correctly handling network byte ordering (`htons`/`ntohs`) for multi-byte `int16_t` chip streams.
- *Manageable Aspects:* Reusing the 65-byte Ethernet frame layout and modular file I/O routines from previous assignments provided a rock-solid foundation.

9.4 Suggested Improvements

1. Extend the simulation to model **asynchronous CDMA** using pseudorandom Gold codes or m-sequences, analyzing how small non-zero cross-correlations introduce multi-user interference noise.
2. Incorporate simulated channel impairments (such as Additive White Gaussian Noise (AWGN) and multipath fading) to evaluate Bit Error Rate (BER) versus Signal-to-Noise Ratio (E_b/N_0).

10 Sources Used

1. Kurose, J. F., and Ross, K. W., *Computer Networking: A Top-Down Approach*, 8th ed., Pearson, 2021.
2. Tanenbaum, A. S., and Wetherall, D. J., *Computer Networks*, 5th ed., Prentice Hall, 2011.
3. Stallings, W., *Data and Computer Communications*, 10th ed., Pearson, 2014.
4. Viterbi, A. J., *CDMA: Principles of Spread Spectrum Communication*, Addison-Wesley, 1995.
5. Proakis, J. G., and Salehi, M., *Digital Communications*, 5th ed., McGraw-Hill, 2008.
6. IEEE Standard for Ethernet (*IEEE Std 802.3-2022*) and POSIX / BSD Socket API Reference Documentation.
7. AI Assistance and Antigravity: Artificial intelligence assistance (Gemini) was utilized for statistical aggregation, test runner orchestration, and generating native TikZ architecture and PGFPlots visualization code. The Antigravity development environment was used to compile native C binaries, execute live socket test suites, and format the comprehensive LaTeX report.